Operating System Capstone

Lab 4: Exception and Interrupt

Ryan, Nody

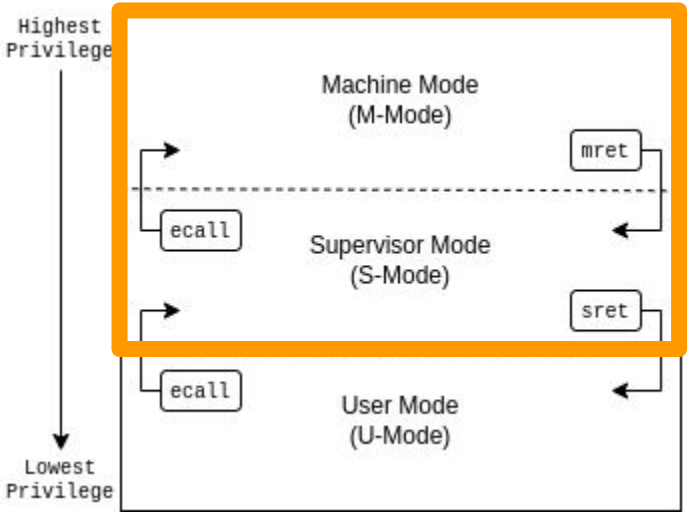
Overview

You have 2 in-class exercises on this lab

- Exercise 4-1: Privilege Level Switching and Exception Handling
- Exercise 4-2: PLIC and UART Interrupts

Privilege Levels

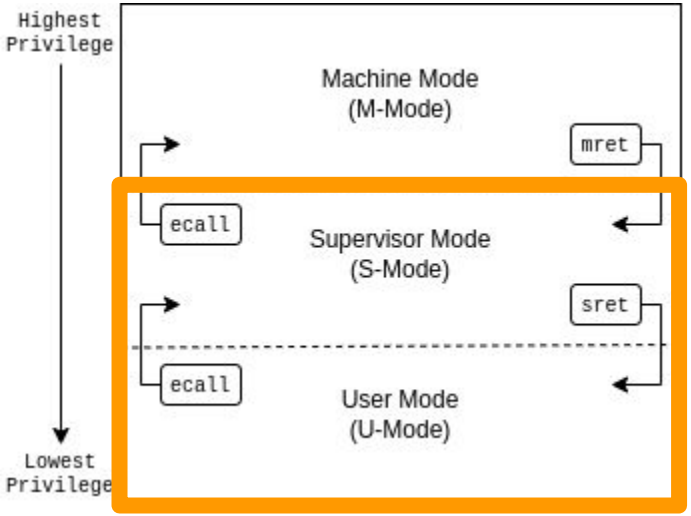
RISC-V defines different privilege levels to protect the OS from user programs.



Goal: Switch from Kernel (S-mode) to User Program (U-mode), and trap back to Kernel (S-mode) when an exception or ecall occurs.

Privilege Levels

RISC-V defines different privilege levels to protect the OS from user programs.



Goal: Switch from Kernel (S-mode) to User Program (U-mode), and trap back to Kernel (S-mode) when an exception or ecall occurs.

Important CSRs

To switch modes and handle traps, we must manipulate these S-mode CSRs:

1. sstatus: Contains the SPP bit. We set this to 0 to return to U-mode.
2. sepc: Return address after the trap
3. scause: Reason for the trap (e.g., 8 means ecall from U-mode).
4. stvec: Points to our trap handler function

Trap Handling Flow

When a User Program executes an ecall instruction:

1. Hardware Action: CPU switches to S-mode, saves the current PC to sepc, sets scause, and jumps to the address in stvec (start.S).
2. Assembly (start.S): Allocates stack space and saves all 32 general-purpose registers into a Trap Frame in memory.
3. C Handler (trap.c): do_trap is called to handle the exception (e.g., read CSRs and print logs).
4. Assembly (start.S): Restores all 32 registers from the Trap Frame.
5. Return: Executes the sret instruction, dropping privileges back to U-mode and resuming execution at sepc.

Exercise 4-1: TODOs and Expected Result

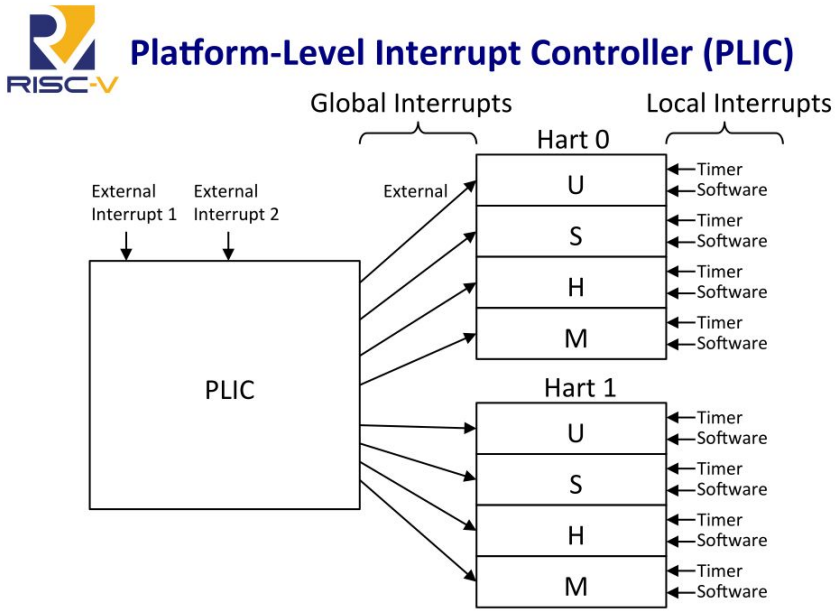
1. Understand handle_exception in Start.S
2. Define the trap frame and implement do_trap to print sepc and scause
3. Finish exec in main.c

Here is the expected result: (sepc may vary)

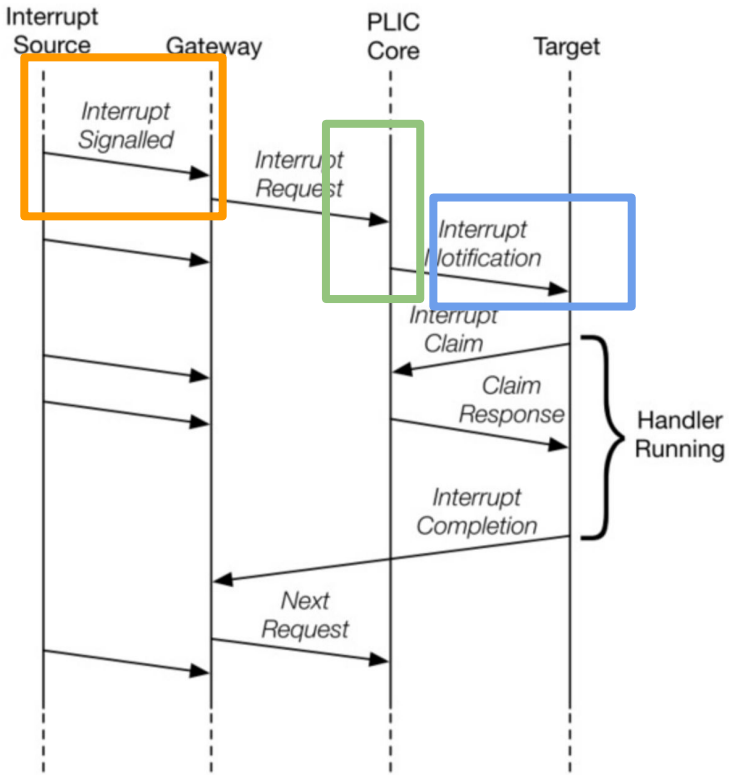
```
Starting kernel ...
sepc: 0x000000008820007a, scause: 0x0000000000000008
sepc: 0x000000008820007a, scause: 0x0000000000000008
sepc: 0x000000008820007a, scause: 0x0000000000000008
sepc: 0x000000008820007a, scause: 0x0000000000000008
sepc: 0x000000008820007a, scause: 0x0000000000000008
```

Platform-Level Interrupt Controller (PLIC)

- Aggregates, prioritizes, and routes external interrupts (e.g., UART) to specific CPU cores (Harts).



PLIC Working Flow



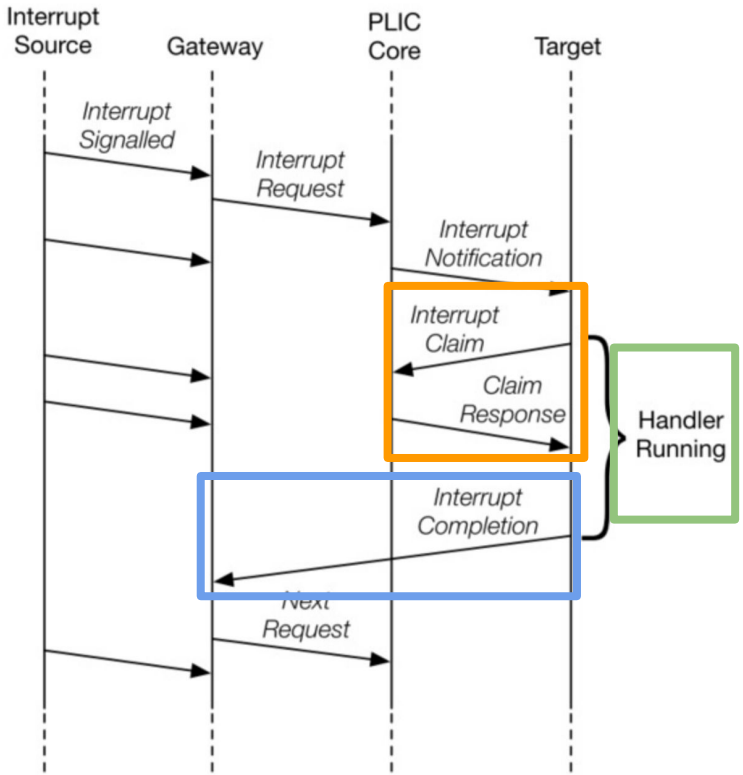
1. External device trigger interrupt

2. PLIC Arbitration:

- priority > 0
- Interrupt Enabled == 1 (for target hart)
- Priority > Threshold

3. PLIC notifies the kernel

PLIC Working Flow



4. PLIC_CLAIM:

- Kernel reads the highest-priority IRQ number.
- PLIC automatically marks the IRQ as "claimed" to prevent duplicate handling.

5. Handler Running

6. PLIC_COMPLETE:

- The Kernel writes the processed IRQ number back to the PLIC Claim/Complete register.
- The Gateway unlocks the interrupt source, allowing future interrupts from this device to be forwarded.

Setting Up PLIC

1. **Interrupt Priority**: Assign a priority level to each IRQ.
2. **Interrupt Enable**: Select which Harts can receive which interrupts.
3. **Priority Threshold**: Set the minimum priority level required to trigger an interrupt for a Hart.

UART Interrupt

- **Interrupt Enable Register (IER):** Setting Bit 0 to 1 means that the UART hardware will trigger an interrupt when the receiving buffer receives data.

- **Modem Control Register (MCR):** Setting Bit 3 (OUT2) to 1 allows the UART interrupt signal to be routed to the interrupt controller (PLIC).

Exercise 4-2: TODOs and Expected Result

1. Understand how to set up PLIC in plic_init()
2. Understand how to handle PLIC interrupt in do_trap()
3. Understand how to enable UART interrupt

Here is the expected result: (you should see what you type)

```
Starting kernel ...
hello
osc
```